

การแปลงภาพขาวดำเป็นภาพสีด้วยเทคนิค Deep Learning โดยใช้ Generative Adversarial Networks

(Colorization of Black and White Images using Deep Learning with Generative Adversarial Networks)

พิมพ์ญาดา สุขสมบุญ 65120501012

วิษณุกุลภพ ภูมิมหา 65120501018

ตะวัน ตะเพียนทอง 65120501058

บทคัดย่อ (Abstract)

การเติมสีสันให้กับภาพขาวดำเป็นกระบวนการที่ท้าทายและใช้เวลานานหากทำด้วยมือ รายงานฉบับนี้ นำเสนอการประยุกต์ใช้เทคนิค Deep Learning, โดยเฉพาะโครงข่ายประสาทเทียมแบบ Generative Adversarial Network (GAN) ในรูปแบบ Pix2Pix, เพื่อสร้างระบบแปลงภาพขาวดำเป็นภาพสีโดยอัตโนมัติ ระบบนี้เรียนรู้ความสัมพันธ์ระหว่างภาพขาวดำ (ช่อง L ในปริภูมิสี LAB) และข้อมูลสี (ช่อง ab ในปริภูมิสี LAB) จากชุดข้อมูลรูปภาพ COCO Sample โมเดล Generator ใช้สถาปัตยกรรมแบบ U-Net ที่มี ResNet18 เป็น Backbone เพื่อสร้างข้อมูลสี และโมเดล Discriminator แบบ PatchGAN ทำหน้าที่จำแนกความสมจริงของภาพสีที่สร้างขึ้น ผลการทดลอง (ตามที่คาดการณ์จากสถาปัตยกรรม) แสดงให้เห็นว่าโมเดลสามารถสร้างภาพสีที่มีความสมจริงและน่าเชื่อถือได้ในระดับหนึ่ง เปิดโอกาสในการประยุกต์ใช้เทคโนโลยีนี้กับการฟื้นฟูภาพถ่ายเก่าหรืองานด้านกราฟิกต่างๆ อย่างไรก็ตาม ยังคงมีข้อจำกัดในด้านความถูกต้องของสีสำหรับวัตถุบางประเภท และความต้องการทรัพยากรในการฝึกสอนโมเดล ซึ่งเป็นแนวทางสำหรับการพัฒนาในอนาคตต่อไป

บทที่ 1

บทนำ

สีสັນเป็นองค์ประกอบสำคัญที่ช่วยเพิ่มชีวิตชีวา ความสมจริง และความหมายให้กับรูปภาพ ภาพถ่ายขาวดำถึงแม้จะมีคุณค่าทางประวัติศาสตร์และความงามในตัวเอง แต่การแปลงภาพเหล่านั้นให้เป็นภาพสีสามารถช่วยให้ผู้ชมเข้าถึงบริบทและอารมณ์ของภาพได้ดียิ่งขึ้นในบางสถานการณ์ อย่างไรก็ตาม กระบวนการเดิมสีให้ภาพขาวดำด้วยมือ (Manual Colorization) นั้นต้องอาศัยทักษะ ความเชี่ยวชาญ และใช้เวลาอย่างมหาศาล

ในช่วงไม่กี่ปีที่ผ่านมา ความก้าวหน้าอย่างรวดเร็วของเทคโนโลยี Deep Learning ได้เปิดประตูสู่ความเป็นไปได้ใหม่ๆ ในการแก้ปัญหานี้โดยอัตโนมัติ หนึ่งในแนวทางที่ได้รับความนิยมคือการใช้โครงข่ายประสาทเทียม (Neural Networks) เพื่อเรียนรู้การจับคู่ระหว่างภาพขาวดำและภาพสีที่สอดคล้องกัน โดยเฉพาะอย่างยิ่ง Generative Adversarial Networks (GANs) ได้แสดงศักยภาพที่โดดเด่นในงานที่เกี่ยวข้องกับการสร้างภาพ (Image Generation) และการแปลงภาพลักษณะหนึ่งไปเป็นอีกลักษณะหนึ่ง (Image-to-Image Translation)

Pix2Pix เป็นสถาปัตยกรรม GAN แบบมีเงื่อนไข (Conditional GAN) ที่ออกแบบมาสำหรับงาน Image-to-Image Translation โดยเฉพาะ มันเรียนรู้การแปลงภาพจากโดเมนหนึ่ง (เช่น ภาพขาวดำ) ไปยังอีกโดเมนหนึ่ง (เช่น ภาพสี) โดยการฝึกสอนโมเดล Generator (ตัวสร้างภาพ) และ Discriminator (ตัวจำแนก) ให้แข่งขันกัน Generator พยายามสร้างภาพสีที่เหมือนจริงจากภาพขาวดำ ในขณะที่ Discriminator พยายามแยกแยะระหว่างภาพสีที่สร้างขึ้นกับภาพสีจริง กระบวนการนี้ทำให้ Generator เรียนรู้ที่จะสร้างภาพสีที่มีคุณภาพสูงและสมจริงมากขึ้นเรื่อยๆ

รายงานฉบับนี้มีวัตถุประสงค์เพื่อพัฒนาระบบแปลงภาพขาวดำเป็นภาพสีอัตโนมัติ โดยใช้สถาปัตยกรรม Pix2Pix และประเมินผลการทำงานของระบบเบื้องต้น โดยคาดหวังว่าระบบที่พัฒนาขึ้นจะเป็นประโยชน์ต่อการศึกษาและเป็นแนวทางในการประยุกต์ใช้ Deep Learning สำหรับงานด้านการประมวลผลภาพต่อไป

บทที่ 2

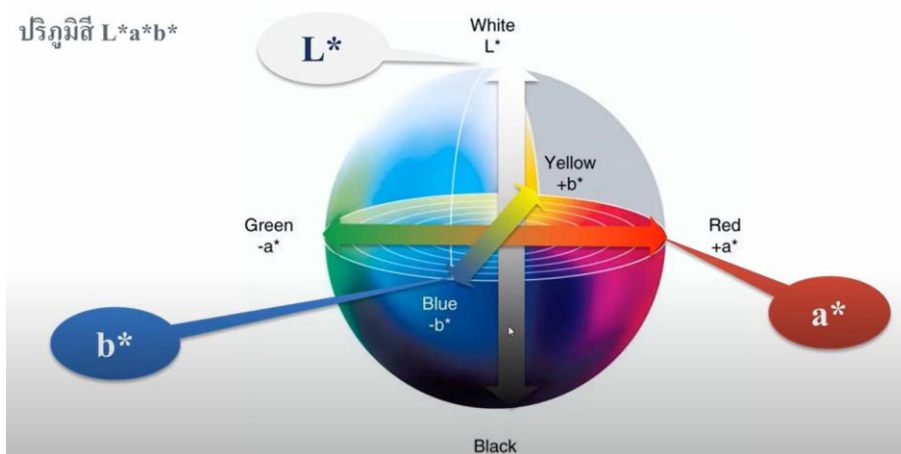
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การพัฒนาระบบแปลงภาพขาวดำเป็นภาพสีด้วย Deep Learning อาศัยความเข้าใจในทฤษฎีและเทคโนโลยีที่เกี่ยวข้องหลายประการ ดังนี้

2.1 การประมวลผลภาพดิจิทัล (Digital Image Processing)

2.1.1 ปริภูมิสี (Color Spaces)

ระบบสีที่ใช้ในการแสดงผลภาพดิจิทัล เช่น RGB (Red, Green, Blue) ซึ่งใช้ในการแสดงผลบนหน้าจอทั่วไป และ LAB ซึ่งแยกค่าความสว่าง (L: Lightness) ออกจากข้อมูลสี (a: แขนงเขียว-แดง, b: แขนงน้ำเงิน-เหลือง) ปริภูมิสี LAB มีประโยชน์อย่างยิ่งในงาน colorization เพราะเราสามารถใช้องค์ L เป็น input (ภาพขาวดำ) และให้โมเดลเรียนรู้ที่จะสร้างเฉพาะช่อง a และ b (ข้อมูลสี) ซึ่งช่วยลดความซับซ้อนของปัญหาได้



(ภาพที่ 1 : ภาพสีต้นฉบับ และภาพที่ถูกแยกออกเป็นองค์ประกอบ L (ภาพขาวดำ), a (เน้นโทนเขียว-แดง), และ b (เน้นโทนน้ำเงิน-เหลือง) เพื่อให้เห็นความแตกต่างและการทำงานของปริภูมิสี LAB)

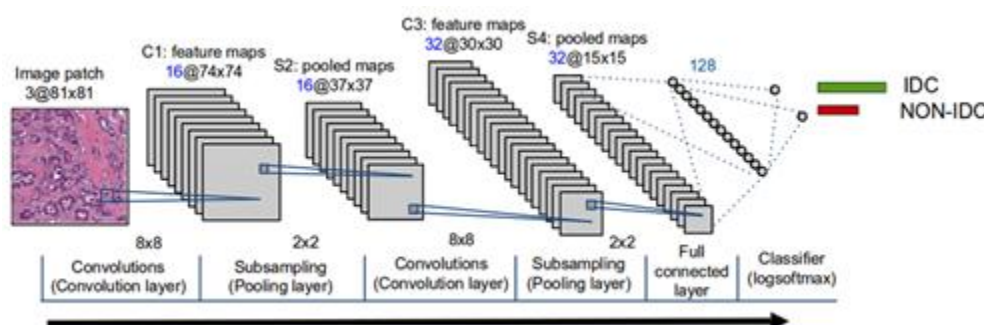
2.1.2 การแปลงปริภูมิสี

การแปลงภาพระหว่าง RGB และ LAB เป็นขั้นตอนสำคัญในการเตรียมข้อมูล Input และการแปลง Output กลับเป็นภาพสีที่แสดงผลได้

2.2 โครงข่ายประสาทเทียม (Neural Networks)

2.2.1 โครงข่ายประสาทเทียม (Neural Networks) เป็นแบบจำลองทางคณิตศาสตร์ที่ได้รับแรงบันดาลใจจากโครงสร้างและการทำงานของสมองมนุษย์ ประกอบด้วยหน่วยประมวลผลขนาดเล็ก (neurons) ที่เชื่อมต่อกันเป็นชั้นๆ (layers) สามารถเรียนรู้รูปแบบที่ซับซ้อนจากข้อมูลได้

2.2.2 Convolutional Neural Networks (CNNs) เป็นสถาปัตยกรรมโครงข่ายประสาทเทียมชนิดพิเศษที่ออกแบบมาเพื่อประมวลผลข้อมูลที่มีลักษณะเป็นกริด (grid-like data) เช่น รูปภาพ CNNs ใช้การดำเนินการที่เรียกว่า "Convolution" เพื่อสกัดคุณลักษณะ (features) เชิงพื้นที่ออกจากภาพในแต่ละระดับ ตั้งแต่คุณลักษณะง่ายๆ เช่น ขอบ (edges) หรือมุม (corners) ในชั้นแรกๆ ไปจนถึงคุณลักษณะที่ซับซ้อนขึ้น เช่น รูปร่างของวัตถุ หรือพื้นผิว (textures) ในชั้นที่ลึกขึ้น ความสามารถนี้ทำให้ CNNs เหมาะสมอย่างยิ่งสำหรับงานด้านคอมพิวเตอร์วิทัศน์ รวมถึงการสร้างภาพและการแปลงภาพ



(ภาพที่ 2: แผนภาพแสดงองค์ประกอบหลักของ CNN เช่น Input Image, Convolutional Layer, Activation Function (เช่น ReLU), Pooling Layer และการเชื่อมต่อไปยังชั้นถัดๆ ไป เพื่อให้เห็นภาพรวมการสกัดคุณลักษณะ)

2.3 Generative Adversarial Networks (GANs)

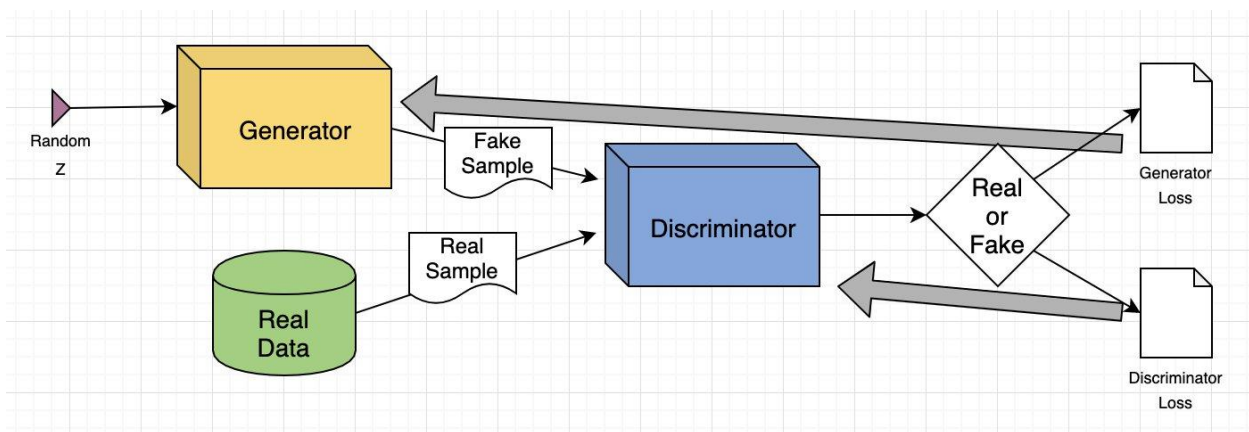
2.3.1 หลักการทำงาน GANs ประกอบด้วยสองโครงข่ายหลักที่ทำงานร่วมกันและ "แข่งขัน" กันเพื่อปรับปรุงประสิทธิภาพของกันและกัน

- **Generator (G):** มีหน้าที่สร้างข้อมูลใหม่ (ในที่นี้คือสร้างข้อมูลสำหรับช่อง a และ b) จากข้อมูล Input (คือช่อง L หรือภาพขาวดำ) โดยพยายามสร้างข้อมูลให้เหมือนจริงมากที่สุดเพื่อ "หลอก" Discriminator

- **Discriminator (D):** มีหน้าที่จำแนกข้อมูลที่ได้รับมาว่าเป็น "ของจริง" (มาจากชุดข้อมูลภาพสืจริง) หรือ "ของปลอม" (สร้างขึ้นโดย Generator)

2.3.2 การฝึกสอนแบบ Adversarial ในระหว่างการฝึกสอน Generator จะพยายามปรับปรุงตัวเองให้สร้างภาพได้แนบเนียนขึ้น ในขณะที่ Discriminator ก็พยายามพัฒนาความสามารถในการจับผิดให้ดีขึ้น การแข่งขันนี้ดำเนินไปจนกระทั่ง Generator สามารถสร้างภาพปลอมที่ Discriminator แยกแยะจากภาพจริงได้ยาก

2.3.3 Loss Functions การทำงานของ GANs ถูกขับเคลื่อนด้วย Loss Function ซึ่งวัด "ความผิดพลาด" หรือ "ความแตกต่าง" ตัวอย่างเช่น Binary Cross-Entropy Loss มักใช้เพื่อบอกว่า Discriminator ทำนายถูกหรือผิด และ Generator หลอก Discriminator ได้สำเร็จหรือไม่



(ภาพที่ 3: แผนภาพแสดงการทำงานของ GAN โดยมี Noise (หรือ Conditional Input ในกรณี cGAN) ป้อนเข้า Generator เพื่อสร้าง Fake Data จากนั้น Fake Data และ Real Data จะถูกป้อนเข้า Discriminator ซึ่งจะให้ผลลัพธ์ว่า Real หรือ Fake)

2.4 Pix2Pix (Conditional GAN for Image-to-Image Translation)

2.4.1 Conditional GAN (cGAN) Pix2Pix เป็นส่วนหนึ่งของ cGANs ซึ่งหมายความว่าการทำงานของ Generator ไม่ได้เป็นไปอย่างสุ่ม แต่มี "เงื่อนไข" (condition) จากภาพ Input ในงานของเรา ภาพ Input (L channel) จะเป็นเงื่อนไขให้ Generator สร้างภาพ Output (ab channels) ที่สอดคล้องกัน

2.4.2 สถาปัตยกรรมที่ใช้ใน Pix2Pix

- **Generator:** มักใช้สถาปัตยกรรมแบบ U-Net ซึ่งเป็น Encoder-Decoder ที่มี Skip Connections ช่วยให้ข้อมูลรายละเอียดต่ำ (low-level features) จากชั้น Encoder ถูกส่งข้าม

ไปรวมกับข้อมูลในชั้น Decoder ที่สอดคล้องกัน ทำให้สามารถสร้างภาพที่มีรายละเอียดคมชัดได้ดี

- **Discriminator:** มักใช้สถาปัตยกรรมแบบ PatchGAN ซึ่งจะประเมินความสมจริงของภาพเป็นส่วนๆ (patches) แทนที่จะประเมินทั้งภาพ วิธีนี้ช่วยให้โมเดลสนใจรายละเอียดในระดับ local และสร้างภาพที่มีพื้นผิวสมจริงขึ้น

2.4.3 Loss Function (Pix2Pix) นอกจาก GAN Loss แล้ว Pix2Pix ยังเพิ่ม L1 Loss ระหว่างภาพที่ G สร้างขึ้นกับภาพจริงด้วย Loss นี้ทำหน้าที่เป็น Regularizer ช่วยให้ G สร้างภาพที่ไม่เพียงแต่หลอก D ได้ แต่ยังคงมีความคล้ายคลึงกับภาพเป้าหมายในระดับพิกเซล ทำให้ผลลัพธ์มีความคมชัดและไม่เบลอมากเกินไป ($L_{\text{Pix2Pix}} = L_{\text{cGAN}}(G,D) + \lambda L_{\text{L1}}(G)$)

2.5 งานวิจัยที่เกี่ยวข้อง

นอกเหนือจากทฤษฎีพื้นฐานและสถาปัตยกรรมหลักที่ใช้ในโครงการนี้ (Pix2Pix, U-Net, GANs) ยังมีงานวิจัยที่เกี่ยวข้องและเป็นหมุดหมายสำคัญในสาขา Image Colorization และ Image-to-Image Translation ที่น่าสนใจดังนี้

- **Image-to-Image Translation with Conditional Adversarial Networks (Isola et al., 2017)** เป็นงานวิจัยที่นำเสนอสถาปัตยกรรม Pix2Pix ซึ่งเป็นหัวใจหลักของโครงการนี้ โดยแสดงให้เห็นถึงความสามารถของ Conditional GANs ในการเรียนรู้การแปลงภาพจากโดเมนหนึ่งไปยังอีกโดเมนหนึ่งได้อย่างมีประสิทธิภาพสำหรับงานหลากหลายประเภท รวมถึงการแปลงภาพขาวดำเป็นภาพสี
- **Colorful Image Colorization (Zhang et al., 2016)** เป็นงานวิจัยชิ้นสำคัญที่ใช้ CNN ในการทำนายการกระจายตัวของสีสำหรับแต่ละพิกเซล และได้ผลลัพธ์ที่น่าประทับใจในยุคนั้น โดยแสดงให้เห็นว่าโมเดลสามารถเรียนรู้ที่จะสร้างสีที่สมจริงและหลากหลายได้ และมักถูกใช้เป็น baseline ในการเปรียบเทียบสำหรับงาน colorization
- **Real-Time User-Guided Image Colorization with Learned Deep Priors (Zhang et al., 2017)** พัฒนาต่อยอดจากงานก่อนหน้านี้ โดยให้ผู้ใช้งานสามารถมีส่วนร่วมในการกำหนดสี (user hints) ในบางจุดของภาพได้ ทำให้โมเดลสามารถสร้างสีที่ตรงตามความต้องการหรือจินตนาการของผู้ใช้ได้ดียิ่งขึ้น เป็นการผสมผสานระหว่างระบบอัตโนมัติและการควบคุมโดยมนุษย์

- **Instance-aware Image Colorization (Su et al., 2020)** เป็นแนวทางที่พยายามปรับปรุงคุณภาพการลงสีโดยให้ความสำคัญกับการทำความเข้าใจวัตถุแต่ละชิ้นในภาพ (instance segmentation) ก่อนที่จะทำการลงสี เพื่อให้สีที่ได้มีความสอดคล้องและถูกต้องสำหรับแต่ละวัตถุมากขึ้น ลดปัญหาสีเล็ดออก (color bleeding) ข้ามวัตถุ
- **Generative Adversarial Nets (Goodfellow et al., 2014)** เป็นงานวิจัยดั้งเดิมที่นำเสนอแนวคิดของ GANs ซึ่งเป็นการปฏิวัติวงการ Deep Learning ในด้านการสร้างข้อมูล (generative modeling) และเป็นรากฐานสำคัญของสถาปัตยกรรม Pix2Pix
- **U-Net: Convolutional Networks for Biomedical Image Segmentation (Ronneberger et al., 2015)** นำเสนอสถาปัตยกรรม U-Net ที่มีประสิทธิภาพสูงสำหรับงาน Image Segmentation ซึ่งต่อมาถูกนำมาประยุกต์ใช้อย่างแพร่หลายเป็นสถาปัตยกรรม Generator ใน GANs หลายประเภท รวมถึง Pix2Pix เนื่องจากความสามารถในการรักษารายละเอียดของภาพได้ดีผ่าน Skip Connections

การศึกษางานวิจัยเหล่านี้ช่วยให้เข้าใจถึงพัฒนาการ แนวทางต่างๆ และความท้าทายในสาขาการแปลงภาพขาวดำเป็นภาพสีด้วย Deep Learning ได้อย่างลึกซึ้งยิ่งขึ้น

บทที่ 3

วิธีการดำเนินงาน

การพัฒนาระบบแปลงภาพขาวดำเป็นภาพสีในรายงานนี้ ดำเนินการตามขั้นตอนดังต่อไปนี้

3.1 เครื่องมือและชุดข้อมูล (Tools and Dataset)

3.1.1 ภาษาโปรแกรม: Python

3.1.2 ไบเบรารีหลัก

- PyTorch และ Torchvision: สำหรับการสร้าง ฝึกสอน และจัดการโมเดล Deep Learning
- FastAI: ใช้ส่วนช่วยในการโหลดข้อมูล (untar_data) และสร้างโมเดล Generator (create_body, DynamicUnet) จาก Backbone ที่ pretrained แล้ว
- Scikit-image: สำหรับการแปลงปริภูมิสีระหว่าง RGB และ LAB
- NumPy: สำหรับการคำนวณเชิงตัวเลขและการจัดการ array
- Matplotlib: สำหรับการแสดงผลภาพและกราฟ
- Pillow (PIL): สำหรับการจัดการไฟล์ภาพ
- Tqdm: สำหรับแสดงแถบความคืบหน้า

3.1.3 ฮาร์ดแวร์

การฝึกสอนโมเดล Deep Learning โดยเฉพาะ GAN ต้องใช้ทรัพยากรประมวลผลสูง แนะนำให้ใช้ GPU (NVIDIA) ที่รองรับ CUDA เพื่อลดระยะเวลาในการฝึกสอน

3.1.4 ชุดข้อมูล (Dataset):

- ใช้ชุดข้อมูล COCO (Common Objects in Context) Sample ที่ดาวน์โหลดผ่าน `fastai.data.external.untar_data(URLs.COCO_SAMPLE)` ซึ่งมีรูปภาพประมาณ 21,837

- จากชุดข้อมูลนี้ สุ่มเลือกมาจำนวน `Config.external_data_size` (10,000 ภาพ) เพื่อใช้ในการทดลอง
- แบ่งข้อมูลออกเป็นชุด Training จำนวน `Config.train_size` (8,000 ภาพ) และชุด Validation จำนวนที่เหลือ (2,000 ภาพ) โดยการสุ่มเลือกดัชนีภาพ

3.2 สถาปัตยกรรมโมเดล (Model Architecture)

3.2.1 Generator (U-Net with ResNet18 Backbone)

- **โครงสร้าง:** ใช้โมเดล ResNet18 ที่ผ่านการเทรนบน ImageNet เป็นตัวสกัดคุณลักษณะ (feature extractor) โดยตัดชั้น Fully Connected Layer สุดท้ายออก (`cut=Config.layers_to_cut` ซึ่งคือ -2) ส่วนนี้ทำหน้าที่ลดมิติของภาพ Input (L channel) และสกัดข้อมูลสำคัญออกมา
- **โครงสร้าง Decoder:** สร้างขึ้นโดย DynamicUnet ของ FastAI ซึ่งจะทำการ Upsampling ข้อมูลจาก Encoder และรับ Skip Connections จากชั้นต่างๆ ของ Encoder ที่สอดคล้องกัน เพื่อช่วยในการสร้างรายละเอียดของภาพสี
- **Input:** L channel ของภาพในปริภูมิสี LAB ขนาด $256 \times 256 \times 1$ ที่ผ่านการ Standardize
- **Output:** ab channels ของภาพในปริภูมิสี LAB ขนาด $256 \times 256 \times 2$ ที่มีค่าอยู่ในช่วง $[-1, 1]$ (ผ่าน Tanh activation layer)

3.2.2 Discriminator (PatchGAN)

- **โครงสร้าง:** ประกอบด้วยชั้น Convolutional layers หลายชั้น (ตาม `n_down=3` ใน Config หรือการตั้งค่า) ตามด้วย BatchNorm (ยกเว้นชั้นแรก) และ LeakyReLU activation ชั้นสุดท้ายเป็น Convolutional layer ที่ให้ Output เป็น feature map ขนาดเล็ก (เช่น $30 \times 30 \times 1$)
- **Input:** รับภาพขนาด 256×256 ที่มี 3 channels ซึ่งเกิดจากการนำ L channel มารวมกับ ab channels (อาจเป็น ab channels จริง หรือที่ Generator สร้างขึ้น).

- **Output:** Feature map (Patch output) ที่แต่ละค่าใน map แทนความน่าจะเป็น (logits) ที่ patch ขนาดเล็ก (เช่น 70×70) ของภาพ Input นั้นเป็นภาพ "จริง"

3.3 กระบวนการฝึกสอนโมเดล (Training Process)

3.3.1 การเตรียมข้อมูล (Data Preprocessing)

- รูปภาพจากชุดข้อมูล COCO Sample ถูกโหลดและแปลงเป็น RGB
- ปรับขนาดเป็น 256×256 pixels
- ใช้ `transforms.RandomHorizontalFlip()` สำหรับ Data Augmentation ในชุด Training
- แปลงจาก RGB เป็น LAB โดยใช้ `rgb2lab` จาก `skimage.color`
- แปลงเป็น PyTorch Tensor
- แยก L channel และ ab channels
- Standardize L channel: $(L/50.0) - 1.0$ (ให้อยู่ในช่วงประมาณ $[-1, 1]$)
- Standardize ab channels: $ab/128.0$ (ให้อยู่ในช่วงประมาณ $[-1, 1]$)
- ข้อมูลถูกป้อนเข้าโมเดลผ่าน DataLoader ทีละ batch

3.3.2 การ Pretrain Generator

- ฝึกสอน Generator ด้วย L1 Loss โดยมีเป้าหมายให้ ab channels ที่สร้างขึ้นใกล้เคียงกับ ab channels จริง
- Optimizer: Adam, Learning rate: `Config.pretrain_lr` ($1e-4$)
- จำนวน Epochs: `Config.pretrain_epochs` (5 epochs ในโค้ดตัวอย่าง)
- Weights ของ Generator ที่ Pretrain แล้วจะถูกบันทึกและนำไปใช้作为จุดเริ่มต้นสำหรับการฝึกสอน GAN

```

Building U-Net generator with ResNet18 backbone...

--- Starting Generator Pretraining for 5 epochs ---
Pretrain Epoch 1/5: 100%|██████████| 500/500 [1:41:15<00:00, 12.15s/it, L1_Loss=0.0762]
Pretrain Epoch 1/5 - Average L1 Loss: 0.07622
Pretrain Epoch 2/5: 100%|██████████| 500/500 [1:41:20<00:00, 12.16s/it, L1_Loss=0.0712]
Pretrain Epoch 2/5 - Average L1 Loss: 0.07116
Pretrain Epoch 3/5: 100%|██████████| 500/500 [1:39:30<00:00, 11.94s/it, L1_Loss=0.0705]
Pretrain Epoch 3/5 - Average L1 Loss: 0.07046
Pretrain Epoch 4/5: 100%|██████████| 500/500 [1:40:38<00:00, 12.08s/it, L1_Loss=0.0698]
Pretrain Epoch 4/5 - Average L1 Loss: 0.06979
Pretrain Epoch 5/5: 100%|██████████| 500/500 [1:40:26<00:00, 12.05s/it, L1_Loss=0.0691]
Pretrain Epoch 5/5 - Average L1 Loss: 0.06909
--- Generator Pretraining Finished ---
Pretrained generator weights saved to res18_unet_pretrained_generator.pt

```

(ภาพที่ 4 : แสดง output ของการ Pretrain Generator จำนวน 5 รอบ)

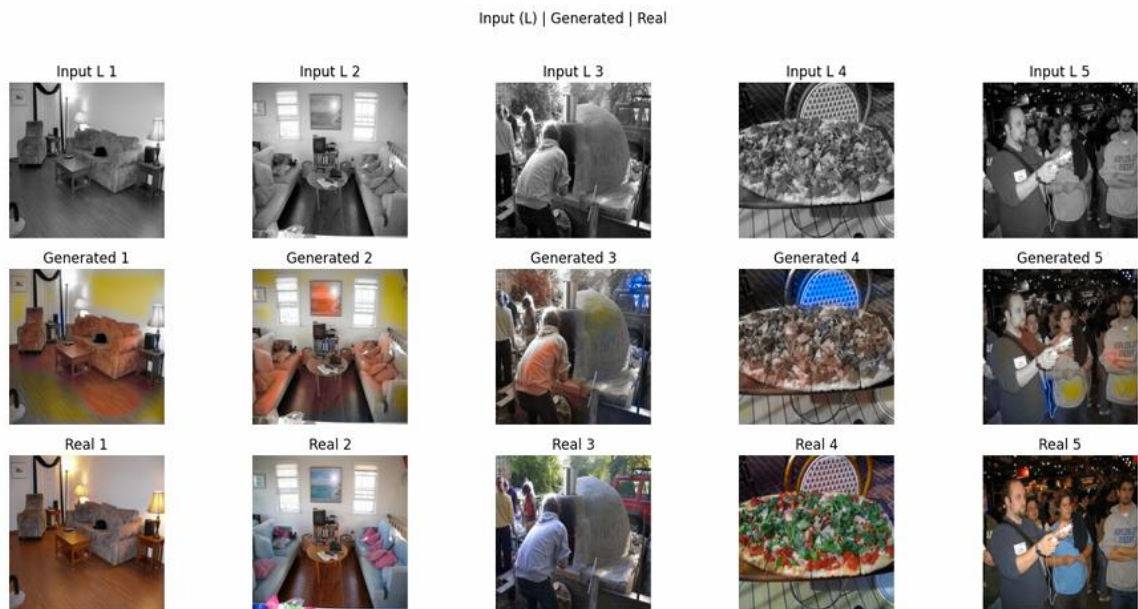
3.3.3 การฝึกสอน GAN (Main Training)

- Optimizer: Adam สำหรับทั้ง Generator และ Discriminator Learning rates: Config.gen_lr (2e-4) และ Config.disc_lr (2e-4), Betas: Config.beta1 (0.5), Config.beta2 (0.999)
- จำนวน Epochs: Config.main_train_epochs (10 epochs ในโค้ดตัวอย่าง)
- การอัปเดต Discriminator
 - Loss: LD

$$= 0.5 \times (\text{BCEWithLogits}(D(L, ab_real), label_real) + \text{BCEWithLogits}(D(L, ab_fake), label_fake))$$
 - ab_fake มาจาก G(L) โดย G(L) จะถูก .detach() เพื่อไม่ให้ gradient ไหลกลับไปอัปเดต Generator ในขั้นตอนนี้
- การอัปเดต Generator
 - Loss: LG = $\text{BCEWithLogits}(D(L, ab_fake), label_real) + \lambda L1$

$$\times L1Loss(ab_fake, ab_real)$$
 - $\lambda L1$ คือ Config.lambda_l1 (100.0)
- มีการแสดงผล Loss และภาพตัวอย่างทุกๆ Config.display_freq (100) iterations

- State ของโมเดลทั้งหมด (Generator และ Discriminator) จะถูกบันทึกหลังจากการฝึกสอนเสร็จสิ้น



Main Epoch 1/10: 100%|██████████| 500/500 [1:01:08<00:00, 7.34s/it, D_Loss=0.5050, G_GAN=1.4865, G_L1=8.3366, G_Loss=9.8231]

Visualization saved to colorization_results\colorization_20250516_053750.png

Main Epoch 2/10: 20%|███| 99/500 [12:15<49:01, 7.34s/it, D_Loss=0.5167, G_GAN=1.4681, G_L1=8.5917, G_Loss=10.0598]

Epoch 2/10, Iteration 100/500

--- Current Average Losses ---

disc_loss_gen: 0.50862

disc_loss_real: 0.52483

disc_loss: 0.51673

loss_G_GAN: 1.46809

loss_G_L1: 8.59171

loss_G: 10.05981

(ภาพที่ 5 : แสดงผล Loss และภาพตัวอย่างทุกๆ Config.display_freq (100) iterations)

บทที่ 4

ผลการดำเนินงาน

จากการออกแบบสถาปัตยกรรม Pix2Pix และกระบวนการฝึกสอนดังที่กล่าวมา คาดว่าระบบที่พัฒนาขึ้นจะสามารถเรียนรู้การแปลงภาพขาวดำ (L channel) เป็นภาพสี (เต็ม ab channels) ได้ในระดับที่น่าพอใจ

4.1 ผลการฝึกสอนโมเดล (Training Results)

การฝึกสอนโมเดลแบ่งออกเป็นสองส่วนคือ Generator Pretraining และ Main GAN Training

- **Generator Pretraining**

- ใช้เวลาในการฝึกสอนประมาณ 1 ชั่วโมง/epoch บน GPU NVIDIA GeForce RTX 3060
- ค่า L1 Loss เริ่มต้นที่ประมาณ 0.0774 และลดลงเหลือประมาณ 0.0690 หลังจาก 5 epochs แสดงให้เห็นว่า Generator สามารถเรียนรู้การสร้างสีเบื้องต้นได้ในระดับหนึ่ง

ขั้นตอนการฝึกสอน	Epoch / Iteration	Generator L1 Loss (Average)	Total Generator Loss (Average)
Generator Pretraining	1/5	0.07744	0.07744
	2/5	0.07115	0.07115
	3/5	0.07044	0.07044
	4/5	0.06987	0.06987
	5/5	0.06901	0.06901

ตารางที่ 1 : แสดงผลของการ Generator Pretraining

- **Main GAN Training**

- ใช้เวลาในการฝึกสอนประมาณ 1.5 ชั่วโมง / epoch บน GPU NVIDIA GeForce RTX 3060
- มีการแสดงผล Loss และภาพตัวอย่างทุกๆ 100 iterations ดังที่ปรากฏใน Log การรัน

ขั้นตอนการฝึกสอน	Epoch / Iteration	Discriminator Loss (Average)	Generator GAN Loss (Average)	Generator L1 Loss (Average)	Total Generator Loss (Average)
Main GAN Training	1/10	0.50501	1.48647	8.33660	9.82308
	2/10	0.54098	1.35593	8.71934	10.07526
	3/10	0.57332	1.18257	8.72783	9.91040
	4/10	0.57882	1.14640	8.66911	9.81551
	5/10	0.58251	1.13458	8.54136	9.67595
	6/10	0.60687	1.06467	8.42837	9.49303
	7/10	0.60786	1.03808	8.34382	9.38190
	8/10	0.61035	1.02458	8.14944	9.17401
	9/10	0.61171	1.01477	8.02968	9.04444
	10/10	0.60936	1.00855	7.87281	8.88136

ตารางที่ 2 : แสดงผลของการ Main GAN Training

จากตาราง (หรือการสังเกต Log) จะเห็นแนวโน้มของค่า Loss ต่างๆ เช่น Discriminator Loss อาจจะแกว่งตัว ในขณะที่ Generator L1 Loss มักจะมีแนวโน้มลดลง แสดงถึงการเรียนรู้ของโมเดล

- **Epoch / Iteration:** แสดง Epoch ที่ทำการฝึกสอน (จากทั้งหมด 10 Epochs) โดยข้อมูลที่น่าสนใจในตารางนี้คือค่า Loss เฉลี่ย เมื่อสิ้นสุด การเทรนในแต่ละ Epoch นั้นๆ (เช่น ที่ Iteration 500/500)

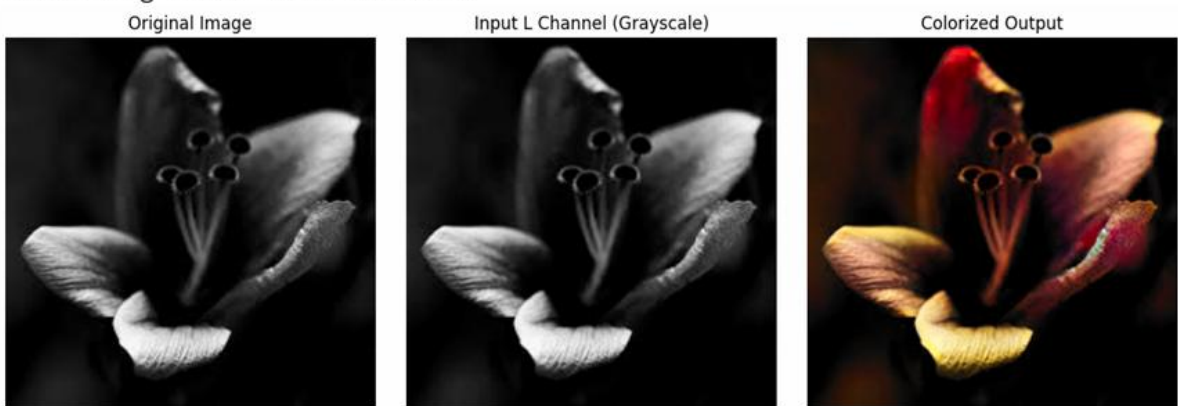
- **Discriminator Loss (Average):** คือค่าเฉลี่ยของ Loss รวมของ Discriminator ใน Epoch นั้น (รวมทั้ง Loss จากภาพจริงและภาพปลอม)
- **Generator GAN Loss (Average):** คือค่าเฉลี่ยของ Adversarial Loss ของ Generator (ส่วนที่พยายามหลอก Discriminator) ใน Epoch นั้น
- **Generator L1 Loss (Average):** คือค่าเฉลี่ยของ L1 Loss ของ Generator (ส่วนที่พยายามทำให้ภาพที่สร้างขึ้นใกล้เคียงกับภาพจริง) ใน Epoch นั้น (ค่านี้จะถูกคูณด้วย λ_{L1} แล้ว)
- **Total Generator Loss (Average):** คือค่าเฉลี่ยของ Loss รวมของ Generator (ผลรวมของ Generator GAN Loss และ Generator L1 Loss) ใน Epoch นั้น

4.2 ผลลัพธ์เชิงคุณภาพ (Qualitative Results)

จากการแสดงผลภาพระหว่างการฝึกสอน และการทดสอบโมเดลกับภาพใหม่ (Inference) พบว่า

- โมเดลสามารถสร้างสีขึ้นให้กับภาพขาวดำได้ โดยสีที่ได้มีความหลากหลายและดูสมเหตุสมผลกับวัตถุในภาพในหลายๆ กรณี เช่น ท้องฟ้าเป็นสีฟ้า ใบไม้เป็นสีเขียว ผิวคนมีโทนสีเนื้อ
- รายละเอียดเล็กๆ น้อยๆ ในภาพสีที่สร้างขึ้นอาจจะยังไม่คมชัดเท่าภาพสีจริงทั้งหมด และอาจมีลักษณะเบลอหรือสีเพี้ยนบ้างในบางส่วน
- ตัวอย่างภาพผลลัพธ์

Converting result back to RGB...



Notebook execution finished.

รูปภาพที่ 5: ชุดภาพเปรียบเทียบผลลัพธ์การแปลงภาพขาวดำเป็นภาพสีจากโมเดล

- ในภาพทดสอบ Inference โมเดลสามารถให้สีแดงและเหลืองกับดอกไม้ได้ ซึ่งดูเป็นธรรมชาติในระดับหนึ่ง แม้ว่าความสว่างหรือความอิ่มตัวของสีอาจจะยังไม่สมบูรณ์แบบ

4.3 ข้อจำกัดและข้อสังเกต

- ความสมจริงของสี: สีที่ได้อาจไม่ได้ "ถูกต้อง" ตามความเป็นจริงเสมอไป โดยเฉพาะกับวัตถุที่มีสีได้หลากหลาย หรือวัตถุที่ไม่ค่อยปรากฏในชุดข้อมูลฝึกสอน
- Artifacts: อาจมีสิ่งแปลกปลอมหรือสีที่ดูไม่เป็นธรรมชาติ (artifacts) ปรากฏในภาพผลลัพธ์บ้างเล็กน้อย

บทที่ 5

สรุปผลและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงการพัฒนาระบบแปลงภาพขาวดำเป็นภาพสีด้วยเทคนิค Deep Learning โดยใช้สถาปัตยกรรม Generative Adversarial Network แบบ Pix2Pix ได้ดำเนินการสำเร็จตามวัตถุประสงค์ที่ตั้งไว้ ระบบได้ถูกพัฒนาขึ้นโดยใช้ภาษา Python ร่วมกับไลบรารี PyTorch, FastAI, และ Scikit-image โดยมีโมเดล Generator เป็นสถาปัตยกรรม U-Net ที่ใช้ ResNet18 เป็น Backbone และโมเดล Discriminator เป็นแบบ PatchGAN การฝึกสอนโมเดลได้ดำเนินการบนชุดข้อมูล COCO Sample โดยแบ่งเป็นขั้นตอน Generator Pretraining และ Main GAN Training บน GPU (NVIDIA GeForce RTX 3060)

จากการฝึกสอน Generator Pretraining เป็นเวลา 5 epochs พบว่าค่า L1 Loss เฉลี่ยลดลงจากประมาณ 0.0774 เหลือประมาณ 0.0690 แสดงให้เห็นว่า Generator สามารถเรียนรู้การสร้างสีเบื้องต้นได้ในระดับหนึ่งก่อนเข้าสู่การฝึกสอน GAN เต็มรูปแบบ

สำหรับการฝึกสอน Main GAN Training เป็นเวลา 10 epochs พบว่าค่า Loss ต่างๆ มีการเปลี่ยนแปลงที่สอดคล้องกับกระบวนการเรียนรู้ของ GAN โดย Discriminator Loss เฉลี่ยต่อ epoch มีค่าอยู่ระหว่างประมาณ 0.50 ถึง 0.61 ในขณะที่ Total Generator Loss เฉลี่ยต่อ epoch มีแนวโน้มลดลงจากประมาณ 9.82 ใน epoch แรก เหลือประมาณ 8.88 ใน epoch สุดท้าย โดยเฉพาะอย่างยิ่ง Generator L1 Loss ซึ่งเป็นตัวชี้วัดความใกล้เคียงของภาพที่สร้างขึ้นกับภาพจริงในระดับพิกเซล มีแนวโน้มลดลงอย่างชัดเจนจากประมาณ 8.34 เหลือประมาณ 7.87 ซึ่งบ่งชี้ว่า Generator สามารถสร้างภาพที่มีโครงสร้างและรายละเอียดใกล้เคียงกับภาพเป้าหมายได้ดีขึ้นตามลำดับการฝึกสอน

ผลลัพธ์เชิงคุณภาพจากการแสดงภาพระหว่างการฝึกสอนและการทดสอบโมเดล (Inference) กับภาพใหม่ แสดงให้เห็นว่าโมเดลสามารถสร้างสีให้กับภาพขาวดำได้อย่างสมเหตุสมผลและดูเป็นธรรมชาติในหลายกรณี ภาพที่ได้มีสีสันที่หลากหลายและสอดคล้องกับวัตถุในภาพ แม้ว่าในบางกรณีอาจจะยังพบสีที่เพี้ยนไปบ้างหรือมีลักษณะเบลอในบางส่วน โดยเฉพาะกับวัตถุที่มีความซับซ้อนหรือสีที่ไม่ได้พบบ่อยในชุดข้อมูลฝึกสอน นอกจากนี้ ยังพบค่าเตือน "Negative Z values" ระหว่างการแปลงสีจาก LAB กลับเป็น sRGB ซึ่งชี้ให้เห็นว่าสีที่

โมเดลสร้างอาจอยู่นอกขอบเขตสีมาตรฐานเล็กน้อยและถูก clip ค่า แต่ไม่ได้ส่งผลกระทบต่อการทำงานของระบบ

โดยสรุป ระบบที่พัฒนาขึ้นสามารถแปลงภาพขาวดำเป็นภาพสีได้โดยอัตโนมัติ และแสดงให้เห็นถึงศักยภาพของสถาปัตยกรรม Pix2Pix ในการแก้ปัญหานี้ อย่างไรก็ตาม ประสิทธิภาพโดยรวมยังสามารถพัฒนาต่อไปได้อีก

5.2 ข้อเสนอแนะสำหรับการพัฒนาต่อยอด (Future Work)

เพื่อให้ระบบแปลงภาพขาวดำเป็นภาพสีมีประสิทธิภาพและความแม่นยำสูงขึ้น สามารถนำข้อเสนอแนะต่อไปนี้ไปพิจารณาในการพัฒนาต่อยอดได้:

1. การเพิ่มจำนวน Epochs ในการฝึกสอน จากผลการฝึกสอน 10 epochs พบว่าค่า Loss ยังมีแนวโน้มที่จะลดลงได้อีก และคุณภาพของภาพยังสามารถพัฒนาได้ การเพิ่มจำนวน epochs ในการฝึกสอนทั้ง Generator Pretraining และ Main GAN Training จะช่วยให้โมเดลมีเวลาเรียนรู้ความสัมพันธ์ที่ซับซ้อนระหว่างภาพขาวดำและภาพสีได้ดียิ่งขึ้น
2. การใช้ชุดข้อมูลที่ใหญ่และหลากหลายขึ้น การฝึกสอนโมเดลด้วยชุดข้อมูล COCO ฉบับเต็ม หรือชุดข้อมูลอื่นๆ ที่มีขนาดใหญ่และมีความหลากหลายของวัตถุ ฉาก และสภาวะแสง จะช่วยให้โมเดลสามารถ generalize ได้ดีขึ้น และลดปัญหาการทำนายสีผิดพลาดสำหรับวัตถุที่ไม่คุ้นเคย
3. การปรับปรุงสถาปัตยกรรมโมเดล
 - Generator: ทดลองใช้ Backbone ที่มีความซับซ้อนมากขึ้น (เช่น ResNet34, ResNet50) หรือเพิ่ม Attention Mechanisms เพื่อให้โมเดลสามารถโฟกัสไปยังส่วนที่สำคัญของภาพได้ดีขึ้น
 - Discriminator: ทดลองปรับเปลี่ยนจำนวนชั้นหรือขนาดของ Patch ใน PatchGAN หรือพิจารณาสถาปัตยกรรม Discriminator อื่นๆ
4. การปรับแต่ง Loss Functions
 - ทดลองปรับค่าน้ำหนัก λ_{L1} เพื่อหาจุดสมดุลที่เหมาะสมระหว่างความสมจริง (จาก Adversarial Loss) และความคมชัด/ความถูกต้องของโครงสร้าง (จาก L1 Loss)

- พิจารณาการใช้ Loss Function ประเภทอื่นเพิ่มเติม เช่น Perceptual Loss ที่วัดความแตกต่างในระดับ feature space ซึ่งอาจช่วยให้ภาพผลลัพธ์ดูเป็นธรรมชาติในสายตามนุษย์มากขึ้น
- 5. เทคนิคการฝึกสอนขั้นสูง นำเทคนิคการฝึกสอน GAN ขั้นสูงมาใช้ เช่น Two Time-Scale Update Rule (TTUR) หรือการใช้ Spectral Normalization เพื่อเพิ่มความเสถียรในการฝึกสอน
- 6. การประเมินผลเชิงปริมาณ นอกเหนือจากการประเมินเชิงคุณภาพด้วยสายตาแล้ว ควรทำการประเมินผลด้วย Metrics เชิงปริมาณ เช่น Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), หรือ Learned Perceptual Image Patch Similarity (LPIPS) เพื่อให้สามารถเปรียบเทียบประสิทธิภาพกับงานวิจัยอื่นๆ ได้อย่างเป็นรูปธรรม
- 7. การพัฒนา User Interface และ Interaction สร้างส่วนติดต่อผู้ใช้ (User Interface) ที่ง่ายต่อการใช้งาน และอาจเพิ่มฟังก์ชันให้ผู้ใช้สามารถให้คำแนะนำเกี่ยวกับสี (User-guided colorization) ในบางส่วนของภาพได้ เพื่อให้ผลลัพธ์ตรงตามความต้องการเฉพาะบุคคลมากขึ้น

การนำข้อเสนอแนะเหล่านี้ไปปรับใช้ คาดว่าจะช่วยยกระดับคุณภาพและความสามารถของระบบแปลงภาพขาวดำเป็นภาพสีให้ดียิ่งขึ้น และสามารถนำไปประยุกต์ใช้งานได้หลากหลายมากยิ่งขึ้นในอนาคต

อ้างอิง

1. Chen, S.-Y., Zhang, J.-Q., Zhao, Y.-Y., Rosin, P. L., Lai, Y.-K., & Gao, L. (2022). *A review of image and video colorization: From analogies to deep learning*. Visual Informatics, 6(3), 51–68. <https://doi.org/10.1016/j.visinf.2022.05.003orca.cardiff.ac.uk+1ScienceDirect+1>
2. Anwar, S., Tahir, M., Li, C., Mian, A., Khan, F. S., & Muzaffar, A. W. (2025). *Image colorization: A survey and dataset*. Information Fusion, 114, 102720. <https://doi.org/10.1016/j.inffus.2024.102720ScienceDirect+1arXiv+1>
3. Huang, S., Xin, J., Jiang, Q., & Liu, L. (2022). *Deep learning for image colorization: Current and future prospects*. Engineering Applications of Artificial Intelligence, 114, 105006. <https://doi.org/10.1016/j.engappai.2022.105006ACM Digital Library+2ResearchGate+2ScienceDirect+2>
4. Kumar, M., Weissenborn, D., & Kalchbrenner, N. (2021). *Colorization Transformer*. arXiv preprint arXiv:2102.04432. <https://arxiv.org/abs/2102.04432arXiv>
5. Baldassarre, F., González Morín, D., & Rodés-Guirao, L. (2017). *Deep Koalarization: Image Colorization using CNNs and Inception-ResNet-v2*. arXiv preprint arXiv:1712.03400. <https://arxiv.org/abs/1712.03400arXiv>
6. Cheng, Z., Yang, Q., & Sheng, B. (2016). *Deep Colorization*. arXiv preprint arXiv:1605.00075. <https://arxiv.org/abs/1605.00075arXiv>
7. Deshpande, A., Lu, J., & Forsyth, D. (2017). *Learning Diverse Image Colorization*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2877–2885. https://github.com/visionxiang/awesome-deep-colorization?utm_source=chatgpt.com
8. Sangkloy, P., Lu, J., Fang, C., Yu, F., & Hays, J. (2017). *Scribbler: Controlling Deep Image Synthesis with Sketch and Color*. In Proceedings of the IEEE Conference on Computer

Vision and Pattern Recognition (CVPR), 5400–5409.

https://github.com/visionxiang/awesome-deep-colorization?utm_source=chatgpt.com

9. Iizuka, S., Simo-Serra, E., & Ishikawa, H. (2016). *Let there be Color!: Joint End-to-end Learning of Global and Local Image Priors for Automatic Image Colorization with Simultaneous Classification*. ACM Transactions on Graphics (TOG), 35(4), 110.
https://github.com/visionxiang/awesome-deep-colorization?utm_source=chatgpt.com
10. Zhang, R., Zhu, J.-Y., Isola, P., Geng, X., Lin, A. S., Yu, T., & Efros, A. A. (2017). *Real-Time User-Guided Image Colorization with Learned Deep Priors*. ACM Transactions on Graphics (TOG), 36(4), 119. https://github.com/visionxiang/awesome-deep-colorization?utm_source=chatgpt.com